project_12, where 12 is the ID of the newly created project. That outer HTML element also needs to contain an inner HTML element with a DOM class name.

You can augment assert_select in a number of ways. The selector argument to assert_select can be just an element, as in div, or an element plus a class or ID decoration, as in div.hidden. In the latter case, a matching element must have both the HTML tag and the DOM class or ID. As with other DOM selectors, a dot (.) indicates a DOM class and a hash mark (#) indicates a DOM ID. You can also use brackets to indicate arbitrary HTML attributes, as in input[name='email']. All this code has been pulled out into a gem called Rails::Dom::Testing.[5]

You can make a more specific match by including a second argument.

Value Type	Assertion Passes If
True	At least one element matches the selector
False	No elements match the selector
Integer	Exactly the integer number of elements match the selector
Range	The number of elements that match the selector is in the range
Regular expression	The text of a matching element matches the Regex
String	The text of a matching element equals the string

You can also use keyword arguments, meaning that assert_select("li", "Noel") is equivalent to assert_select("li", text: "Noel"). The keyword versions allow you to use multiple assertions in one assert_select call.

Keyword	Assertion Passes If
:count	The number of matching elements equals the argument
:html	A matching element contains the HTML content
:text	A matching element contains the text content
:maximum	The number of matching elements is less than or equal to the argument
:minimum	The number of matching elements is greater than or equal to the argument

5. https://github.com/rails/rails-dom-testing